

PROYECTO No. 1

Motor de Parsing, Validaciones Estructurales y Simulación de Autómatas Finitos Deterministas (AFD)

OBJETIVO GENERAL

Diseñar e implementar un sistema en capaz de cargar, validar y simular un Autómata Finito Determinista (AFD) a partir de su quintúpla formal, procesando archivos de entrada .txt para verificar el reconocimiento de cadenas con su traza paso a paso.

OBJETIVOS DE APRENDIZAJE

- Modelar formalmente un Autómata Finito Determinista (AFD) mediante la representación estricta de su Tupla $M = (Q, \Sigma, \delta, q_0, F)$ utilizando estructuras de datos apropiadas (listas, conjuntos y diccionarios).
- Aplicar expresiones regulares para la construcción de un motor de validación de la sintaxis y extracción de los componentes de la quintúpla a partir de un archivo de texto.
- Validar las propiedades y restricciones teóricas de la quintúpla para garantizar la integridad estructural del AFD antes de su procesamiento.
- Implementar el motor de simulación de un AFD que procese cadenas de entrada paso a paso, generando la traza completa de ejecución y determinando si la cadena pertenece o no al lenguaje reconocido por el autómata.

DESCRIPCIÓN DEL PROBLEMA

El objetivo del proyecto es construir una aplicación en el lenguaje de programación Python modo consola capaz de representar, validar y simular un **Autómata Finito Determinista (AFD)** a partir de su definición formal como quintúpla $M = (Q, \Sigma, \delta, q_0, F)$. El sistema deberá permitir la carga de datos tanto de forma interactiva (consola) como mediante la lectura de un archivo plano de texto (.txt) analizado a través de expresiones regulares.

Funcionalidad 1: Carga

- **Ingreso Interactivo:** Menú dinámico que permita ingresar cada elemento de la quintúpla paso a paso desde la consola.
- **Carga por Archivo (.txt):**

- Estructura del Archivo de Entrada:
- Si alguna línea del archivo no cumple con el patrón de estructura definido, el programa debe señalar el error de sintaxis en el archivo sin colapsar.

Funcionalidad 2: Motor de Validación de la Tupla

Antes de ejecutar cualquier cadena, el sistema debe garantizar la integridad del AFD:

1. **Validación de Estados:** Verificar que $q_0 \in Q$ y que $F \subseteq Q$. Es decir, el estado inicial debe pertenecer al conjunto de estados y comprobar que todos los elementos de F pertenezcan a Q .
2. **Consistencia de Transiciones:** Asegurar que los estados y símbolos involucrados en cada transición pertenezcan a Q y Σ respectivamente.
3. **Verificación Determinista:** Validar que para cada par (*estado, símbolo*) exista exactamente una transición. Si detecta transiciones múltiples o vacías, debe alertar que la entrada corresponde a un AFND y no a un AFD.

Funcionalidad 3: Motor de evaluación y simulación paso a paso

Evaluación individual o por Lote: Opción para ingresar cadenas manuales o leer un conjunto de cadenas a probar.

Traza de ejecución paso a paso: Impresión en consola del recorrido del autómata indicando:

- Estado actual.
- Símbolo procesado.
- Siguiendo estado de transición.
- Veredicto final (*Aceptada* o *Rechazada*) indicando si el estado final pertenece a F .



REQUISITOS FUNCIONALES

El programa deberá presentar un menú con, al menos, las siguientes opciones:

1. Crear un AFD manualmente.
2. Cargar un AFD desde un archivo .txt.
3. Mostrar la definición formal del AFD.
4. Mostrar la tabla de transición.
5. Validar la estructura del autómata.
6. Evaluar una cadena.
7. Evaluar un archivo de cadenas.
8. Consultar el historial de evaluaciones.
9. Cargar o crear otro autómata.
10. Salir.

INGRESO MANUAL

El sistema deberá permitir ingresar:

- Nombre o identificador del autómata.
- Conjunto de estados .
- Alfabeto .
- Estado inicial .
- Conjunto de estados finales .
- Función de transición .

El programa deberá evitar estados o símbolos duplicados.

INGRESO DESDE ARCHIVO

El sistema deberá leer la definición del AFD desde un archivo de texto con una estructura obligatoria, por ejemplo:



```
NOMBRE=AFD_AB
ESTADOS=q0, q1, q2
ALFABETO=a, b
INICIAL=q0
FINALES=q2
TRANSICIONES:
q0, a, q1
q0, b, q0
q1, a, q1
q1, b, q2
q2, a, q1
q2, b, q0
```

EVALUACIÓN DE CADENAS

El programa deberá permitir ingresar una cadena y:

1. Validar que todos sus símbolos pertenezcan al lenguaje.
2. Comenzar en el orden definido.
3. Procesar los símbolos en el orden ingresado.
4. Aplicar la función de transición.
5. Determinar el estado final.
6. Indicar si la cadena fue aceptada o rechazada.

ANÁLISIS ESTRUCTURAL

Después de validarlo, el programa deberá identificar:

- Estados alcanzables desde .
- Estados inaccesibles.
- Estados de aceptación alcanzables.
- Si el lenguaje reconocido podría ser vacío.



Estados alcanzables: q_0, q_1, q_2
Estados inaccesibles: q_3
Estados finales alcanzables: q_2

RESTRICCIONES

El proyecto deberá desarrollarse en **Python** mediante de consola.

- No se permitirá utilizar una biblioteca que implemente directamente autómatas finitos.
- La definición del AFD deberá almacenarse utilizando estructuras apropiadas, como listas, conjuntos y diccionarios.
- La función de transición no podrá implementarse mediante una cadena extensa de if, else o switch para un autómata específico.
- El programa deberá funcionar con cualquier AFD válido que respete el formato establecido.
- La lectura y validación sintáctica del archivo deberá utilizar expresiones regulares.
- El sistema deberá separar, al menos, los componentes de carga, validación y simulación.
- No se permitirá modificar el código fuente para cambiar el autómata evaluado.
- El programa deberá aceptar cualquier cantidad válida de estados y transiciones dentro de los límites establecidos.
- Todos los errores deberán ser controlados mediante mensajes comprensibles; el programa no deberá colapsar.

EVALUACIÓN:

- a. Documento PDF que contiene el análisis y diseño (algoritmo) del programa, el cual debe de contener lo siguiente:
 - Acciones del programa: Enumerar las acciones disponibles.
 - Datos de entrada: Especificar la información que se solicitará al usuario y definir el tipo de dato para cada uno.
 - Variables: Listar las variables a utilizar.
 - Condiciones y cálculos: Describir las restricciones y fórmulas.
 - Algoritmo: Presentar un diagrama de flujo (elaborado en Draw.io) que detalle la lógica completa del programa.
- b. Programa funcional y libre de errores con lo solicitado:

- Documentación interna: El programa debe incluir comentarios claros que expliquen la inicialización del sistema la lógica del menú las validaciones el cálculo de tarifas multas y descuentos y el uso de colores en consola
- Programa en con la funcionalidad solicitada en este documento.

Entregables y fecha:

- Manual de usuario
- Manual técnico
- Código fuente

Fecha de presentación del proyecto: 30 de agosto antes de las 23:59, en el Portal Académico.

CONSIDERACIONES IMPORTANTES

- El proyecto debe ser desarrollado de **parejas**, las cuales serán conformadas por afinidad.
- La funcionalidad del programa debe ser implementada en su totalidad por los alumnos; no es permitido utilizar funciones o módulos que hayan sido realizados, publicados o compartidos por otros autores o entornos de trabajo.
- Asegúrese de gestionar adecuadamente la entrada de datos del usuario y validar lo necesario en cuanto al tipo de datos que se solicita ingresar.
- **Es requerido que los autores de la codificación demuestren el dominio completo de lo implementado para que se le asigne la nota obtenida.**
- El proyecto deberá ser entregado en el espacio habilitado en el portal. Para la calificación se descargará el proyecto entregado por esta vía. No se aceptan entregas vía correo electrónico u otro medio.
- **COPIA PARCIAL O TOTAL DEL PROYECTO TENDRÁ UNA NOTA DE 0 PUNTOS, Y SE NOTIFICARÁ A LA COORDINACIÓN DEL ÁREA PARA QUE SE APLIQUEN LAS SANCIONES CORRESPONDIENTES A LOS ALUMNOS DEL EQUIPO DE TRABAJO.**

POLÍTICA DE USO DE IA Y DOMINIO INDIVIDUAL DEL PROYECTO

El uso de herramientas de Inteligencia Artificial (IA) está permitido únicamente como guía de apoyo o consulta. No sustituye el proceso de análisis, diseño ni programación que debe realizar el estudiante.

Todo lo que se presente en el proyecto debe poder ser explicado, justificado y defendido individualmente por cada integrante del equipo.

Cada estudiante deberá demostrar dominio del proyecto, explicando la lógica del programa, ejecutándolo correctamente.

Si se evidencia que un estudiante:

- No comprende el código desarrollado,
- No puede explicar el análisis o las decisiones implementadas
- demuestra dependencia total del compañero o de herramientas externas.

Esto será considerado incumplimiento de los objetivos formativos del curso y podrá calificarse como plagio o uso indebido de recursos, aplicándose una penalización individual de hasta el 100% de la nota del proyecto, según la gravedad del caso.

